

torch.nn.functional.grid_sample#

[https://pytorch.org/docs/stable/generated/torch.nn.functional.grid_sample.ht](https://pytorch.org/docs/stable/generated/torch.nn.functional.grid_sample.html)

Description:

Given an input and a flow-field grid, computes the output using input values and pixel locations from grid. Currently, only spatial (4-D) and volumetric (5-D) input are supported. In the spatial (4-D) case, for input with shape $(N, C, H_{\text{in}}, W_{\text{in}})$ and grid with shape $(N, H_{\text{out}}, W_{\text{out}}, 2)$, the output will have shape $(N, C, H_{\text{out}}, W_{\text{out}})$. For each output location $\text{output}[n, :, h, w]$, the size-2 vector $\text{grid}[n, h, w]$ specifies input pixel locations x and y , which are used to interpolate the output value $\text{output}[n, :, h, w]$. In the case of 5D inputs, $\text{grid}[n, d, h, w]$ specifies the x, y, z pixel locations for interpolating $\text{output}[n, :, d, h, w]$. `mode` argument specifies nearest or bilinear interpolation method to sample the input pixels.

Function Signature

```
torch.nn.functional.grid_sample(input, grid,  
mode='bilinear', padding_mode='zeros', align_corners=None)  
[source]#
```

Parameters

Returns

Return type

Returns:

output (Tensor)

Notes & Warnings

NOTE

Note This function is often used in conjunction with `affine_grid()` to build Spatial Transformer Networks .

NOTE

Note When using the CUDA backend, this operation may induce nondeterministic behaviour in its backward pass that is not easily switched off. Please see the notes on Reproducibility for background.

NOTE

Note NaN values in grid would be interpreted as -1.

⚠ WARNING

Warning When `align_corners = True`, the grid positions depend on the pixel size relative to the input image size, and so the locations sampled by `grid_sample()` will differ for the same input given at different resolutions (that is, after being upsampled or downsampled). The default behavior up to version 1.2.0 was `align_corners = True`. Since then, the default behavior has been changed to `align_corners = False`, in order to bring it in line with the default for `interpolate()`.

NOTE

Note mode='bicubic' is implemented using the cubic convolution algorithm with $\alpha = -0.75$. The constant α might be different from packages to packages. For example, PIL and OpenCV use -0.5 and -0.75 respectively. This algorithm may “overshoot” the range of values it’s interpolating. For example, it may produce negative values or values greater than 255 when interpolating input in $[0, 255]$. Clamp the results with `torch.clamp()` to ensure they are within the valid range.

Detailed Documentation

Documentation

Given an input and a flow-field grid, computes the output using input values and pixel locations from grid.

Currently, only spatial (4-D) and volumetric (5-D) input are supported.

In the spatial (4-D) case, for input with shape $(N, C, H_{\text{in}}, W_{\text{in}})$ and grid with shape $(N, H_{\text{out}}, W_{\text{out}}, 2)$, the output will have shape $(N, C, H_{\text{out}}, W_{\text{out}})$.

For each output location $\text{output}[n, :, h, w]$, the size-2 vector $\text{grid}[n, h, w]$ specifies input pixel locations x and y , which are used to interpolate the output value $\text{output}[n, :, h, w]$. In the case of 5D inputs, $\text{grid}[n, d, h, w]$ specifies the x, y, z pixel locations for interpolating $\text{output}[n, :, d, h, w]$. `mode` argument specifies nearest or bilinear interpolation method to sample the input pixels.

`grid` specifies the sampling pixel locations normalized by the input spatial dimensions. Therefore, it should have most values in the range of $[-1, 1]$. For example, values $x = -1, y = -1$ is the left-top pixel of input, and values $x = 1, y = 1$ is the right-bottom pixel of input.

If `grid` has values outside the range of $[-1, 1]$, the corresponding outputs are handled as defined by `padding_mode`. Options are

`padding_mode="zeros"`: use 0 for out-of-bound grid locations,

`padding_mode="border"`: use border values for out-of-bound grid locations,

`padding_mode="reflection"`: use values at locations reflected by the border for out-of-bound grid locations. For location far away from the border, it will keep being reflected until becoming in bound, e.g., (normalized) pixel location $x = -3.5$ reflects by border -1 and becomes $x' = 1.5$, then reflects by border 1 and becomes $x'' = -0.5$.

This function is often used in conjunction with `affine_grid()` to build Spatial Transformer Networks .

When using the CUDA backend, this operation may induce nondeterministic behaviour in its backward pass that is not easily switched off. Please see the notes on Reproducibility for background.

NaN values in grid would be interpreted as -1.

input (Tensor) – input of shape $(N, C, H_{\text{in}}, W_{\text{in}})$ (4-D case) or $(N, C, D_{\text{in}}, H_{\text{in}}, W_{\text{in}})$ (5-D case)

grid (Tensor) – flow-field of shape $(N, H_{\text{out}}, W_{\text{out}}, 2)$ (4-D case) or $(N, D_{\text{out}}, H_{\text{out}}, W_{\text{out}}, 3)$ (5-D case)

mode (str) – interpolation mode to calculate output values 'bilinear' | 'nearest' | 'bicubic'. Default: 'bilinear' Note: mode='bicubic' supports only 4-D input. When mode='bilinear' and the input is 5-D, the interpolation mode used internally will actually be trilinear. However, when the input is 4-D, the interpolation mode will legitimately be bilinear.

padding_mode (str) – padding mode for outside grid values 'zeros' | 'border' | 'reflection'. Default: 'zeros'

align_corners (bool, optional) – Geometrically, we consider the pixels of the input as squares rather than points. If set to True, the extrema (-1 and 1) are considered as referring to the center points of the input's corner pixels. If set to False, they are instead considered as referring to the corner points of the input's corner pixels, making the sampling more resolution agnostic. This option parallels the align_corners option in interpolate(), and so whichever option is used here should also be used there to resize the input image before grid sampling. Default: False

When align_corners = True, the grid positions depend on the pixel size relative to the

input image size, and so the locations sampled by `grid_sample()` will differ for the same input given at different resolutions (that is, after being upsampled or downsampled). The default behavior up to version 1.2.0 was `align_corners = True`. Since then, the default behavior has been changed to `align_corners = False`, in order to bring it in line with the default for `interpolate()`.

`mode='bicubic'` is implemented using the cubic convolution algorithm with $\alpha=-0.75$. The constant α might be different from packages to packages. For example, PIL and OpenCV use -0.5 and -0.75 respectively. This algorithm may “overshoot” the range of values it’s interpolating. For example, it may produce negative values or values greater than 255 when interpolating input in `[0, 255]`. Clamp the results with `torch.clamp()` to ensure they are within the valid range.